Noname manuscript No.
(will be inserted by the editor)

Variable neighborhood search: basics and variants

Pierre Hansen · Nenad Mladenović · Raca Todosijević · Saïd Hanafi

Received: date / Accepted: date

Abstract Variable neighborhood search (VNS) is a framework for building heuristics, based upon systematic changes of neighborhoods both in a descent phase, to find a local minimum, and in a perturbation phase to escape from the corresponding valley. In this paper we present some of VNS basic schemes as well as several VNS variants deduced from these basic schemes. In addition, our survey includes parallel implementations and hybrids with other metaheuristics.

Keywords Variable neighborhood search · Metaheuristic · Heuristic

P. Hansen
GERAD & HEC Montréal, 3000 chemin de la Côte-Sainte-Catherine, Montréal (Québec), Canada
E-mail: pierre.hansen@gerad.ca

N. Mladenović · R. Todosijević
LAMIH-UVHC, CNRS UMR 8201, Le Mont Houy Valenciennes, Cedex 9 - France and Mathematical Institute, SANU, Belgrade, Serbia
E-mail: {nenadmladenovic12, racatodosijevic}@gmail.com

S. Hanafi
LAMIH-UVHC, CNRS UMR 8201, Le Mont Houy Valenciennes, Cedex 9 - France
Tel.: +33 3 27 51 19 51
E-mail: said.hanafi@univ-valenciennes.fr

Contents

1 Introduction . . . 3
2 Variable neighborhood search ingredients . . . 4
2.1 Shaking procedure . . . 4
2.2 Neighborhood change step . . . 5
2.3 Improvement procedures within VNS . . . 6
2.3.1 Local search . . . 6
2.3.2 Variable neighborhood descent procedures . . . 7
2.3.2.1 Sequential variable neighborhood descent procedures . . . 8
2.3.2.2 Nested variable neighborhood descent procedures . . . 11
2.3.2.3 Mixed variable neighborhood descent procedures . . . 11
3 Variable neighborhood search variants . . . 13
3.1 Fixed neighborhood search . . . 13
3.2 Basic VNS . . . 13
3.3 Reduced VNS . . . 14
3.4 General VNS . . . 14
3.5 Skewed VNS . . . 16
3.6 Nested VNS . . . 17
4 Advanced VNS variants . . . 17
4.1 Variable neighborhood decomposition search . . . 17
4.2 Primal-dual VNS . . . 18
4.3 VNS for nonlinear optimization . . . 18
4.3.1 Glob-VNS . . . 18
4.3.2 Gauss-VNS . . . 19
4.3.3 VNS with Modified Nelder Mead as a local search . . . 20
4.4 VNS for Mixed Integer Non-Linear Programs . . . 21
4.5 Variable formulation space search . . . 21
4.6 Matheuristics with VNS . . . 23
4.6.1 Variable Neighborhood Branching . . . 24
4.6.2 Hybrid Variable Neighborhood decomposition search heuristics . . . 24
4.6.3 Variable Neighborhood Pump . . . 26
4.6.4 Diving heuristics . . . 27
4.7 Parallel VNS . . . 30
4.8 Hybrids . . . 30
5 Conclusions . . . 30

1 Introduction

An optimization problem may be stated in a general form as:

$$\min\{f(x)|x \in X \subseteq \mathcal{S}\} \tag{1}$$

where $\mathcal{S}, X, x$ and f respectively denote the solution space and the feasible set, a feasible solution and a real-valued objective function. Depending on the set $\mathcal{S}$ we distinguish combinatorial optimization problem (in which the set $\mathcal{S}$ is finite but extremely large) and continuous optimization problems ($\mathcal{S} = \mathbb{R}^n$). Let $\mathcal{N}(x)$ denote a neighborhood structure of a given solution x (i.e., $\mathcal{N} : X \to \mathcal{P}(X)$, where $\mathcal{P}(X)$ denotes the power set of the set X). Generally, a neighborhood $\mathcal{N}(x)$ is defined relative to a given metric (or quasi-metric) function δ introduced in the solution space $\mathcal{S}$ as:

$$\mathcal{N}(x) = \{y \in X|\delta(x, y) \leq \alpha\}$$

where α is a given positive number. Then, on the one hand a solution $x^* \in \mathcal{N}(x)$ is a local minimum, relative to neighborhood $\mathcal{N}(x^*)$, for problem (1) if $f(x^*) \leq f(x),\ \forall x \in \mathcal{N}(x^*)$, on the other hand, a solution $x^* \in X$ is an optimal solution (global optimum) for problem (1) if $f(x^*) \leq f(x),\ \forall x \in X$.

Many practical instances of problems of form (1), cannot be solved exactly (i.e., providing an optimal solution along with the proof of its optimality) in reasonable time. It is well-known that many problems of the form (1) are NP-hard, i.e., no algorithm with a number of steps polynomial in the size of the instances is known for solving any of them and that if one were found it would be a solution for all. Therefore, there is a need for heuristics able to quickly produce an approximate solution of high quality, or sometimes an optimal solution but without proof of its optimality. Note that there could be a very large number of local minima for some optimization problems. Usually, it is not hard to get a local minimum but it is not easy to find a global one. Indeed such search methods may get trapped in a local optimum and miss the global one. Resolving local optima trap problems is one important issue and consists in finding a way to escape from a local minimum within some method.

Variable neighborhood search (VNS) is a metaheuristic proposed by Mladenović and Hansen in 1997 [47]. It represents a flexible framework for building heuristics for approximately solving combinatorial and non-linear continuous optimization problems. VNS systematically changes neighborhood structures during the search for an optimal (or near-optimal) solution based on the following observations:

(*i*) A local optimum relative to one neighborhood structure is not necessarily a local optimum for another neighborhood structure;
(*ii*) A global optimum is a local optimum with respect to all neighborhood structures;
(*iii*) Empirical evidence shows that for many problems all or a large majority of the local optima are relatively close to each other [35].

The first property is exploited by increasingly using complex moves in order to find local optima with respect to all neighborhood structures used. The second property suggests using several neighborhoods, if local optima found are of poor

quality. Finally, the third property suggests increased exploitation of the vicinity of the current incumbent solution.

This survey reviews not only VNS variants already described in the last survey [29] but also new VNS variants proposed in last few years (e.g., cyclic VND, nested VNS, two-level VNS, VNS heuristics for solving mixed integer programs etc.). These new variants have been used for solving many NP hard problems on which they turned out to be new state-of-the-art heuristics and thus applying these variants on other optimization problems leads in promising future work directions. Hence, the main purpose of this survey is to provide practitioners with precise user-guide for creating a VNS based heuristic rather than compare VNS variants or provide exhaustive list of VNS applications.

The rest of the paper is organized as follows. In Section 2 we present the main ingredients of a VNS heuristic. More precisely, we describe the shaking procedure, the most common improvement procedures used within a VNS, such as local search and variable neighborhood descent variants; and neighborhood change functions. Section 3 is dedicated to the basic VNS variants and Section 4 to more advanced VNS variants. Finally, in Section 5, we draw some conclusions.

2 Variable neighborhood search ingredients

The ingredients of a variable neighborhood search heuristic include an *improvement phase* used to possibly improve a given solution and a so-called *shaking phase* used to hopefully resolve local minima traps. The improvement phase and the shaking procedure, together with neighborhood change step are executed alternately until fulfilling a predefined stopping criterion. In other words, the following three steps are repeated until some stopping criterion is fulfilled:

(1) Shaking procedure
(2) Improvement procedure
(3) Neighborhood change step

In order to distinguish neighborhoods used in shaking and improvement procedures, we used two different notations $\mathcal{N}$ and N, respectively.

2.1 Shaking procedure

The aim of a shaking procedure used within a VNS heuristic is to hopefully resolve local minima traps. Let $\mathcal{N} = \{\mathcal{N}_1, \ldots, \mathcal{N}_{k_{max}}\}$ be a set of operators such that each operator $\mathcal{N}_k$, $1 \leq k \leq k_{max}$ maps a given solution x to a predefined neighborhood structure $\mathcal{N}_k(x)$. Note that the order of operators in the set $\mathcal{N}$ will also define the order of examining neighborhood structures of a given solution x. The simple shaking procedure consists in selecting a random solution from the kth neighborhood structure, $\mathcal{N}_k(x)$. For some problem instances, a completely random jump in the kth neighborhood is too diversified. Hence, sometimes it is preferable to do so-called intensified shaking which takes into account how sensitive is the objective function to small changes (shaking) of the solution. However, for the sake of simplicity we will assume that each VNS variant presented hereafter uses a simple shaking procedure based on selecting a random solution from the kth neighborhood structure (see Algorithm 1).

Algorithm 1: Shaking procedure

Function `Shake`$(x, k, \mathcal{N})$;
choose $x' \in \mathcal{N}_k(x)$ at random;
return x'

2.2 Neighborhood change step

The purpose of a neighborhood change step is to guide the variable neighborhood search heuristic while exploring the solution space. More precisely, it makes a decision on which neighborhood will be explored as the next as well as whether some solution will be accepted as a new incumbent solution or not. The widely used neighborhood change procedures are:

- `Sequential neighborhood change step` (see e.g., [66]) - The steps of the sequential neighborhood change step are given in Algorithm 2. If an improvement of the incumbent solution in some neighborhood structure occurs, the search is resumed in the first neighborhood structure (according to the defined order) of the new incumbent solution; otherwise the search is continued in the next neighborhood (according to the defined order).

Algorithm 2: Sequential neighborhood change step

Procedure `Neighborhood_change_sequential`(x, x', k)
if $f(x') < f(x)$ **then**
  $x \leftarrow x'$;
  $k \leftarrow 1$;
else
  $k \leftarrow k + 1$;
end

- `Cyclic neighborhood change step` (see e.g., [63]) - regardless whether there is an improvement with respect to some neighborhood or not, the search is continued in the next neighborhood structure in the list (see Algorithm 3).

Algorithm 3: Cyclic neighborhood change step

Procedure `Neighborhood_change_cyclic`(x, x', k)
$k \leftarrow k + 1$;
if $f(x') < f(x)$ **then**
  $x \leftarrow x'$;
end

- `Pipe neighborhood change step` (see e.g., [65]) - if an improvement of the current solution occurs in some neighborhood, the exploration is continued in that neighborhood (see Algorithm 4).
- `The skewed neighborhood change step` (see e.g., [8]) may accept as new incumbent solution not only improving solution but also non improving one. The purpose of such neighborhood change step is to allow exploration of valleys far

Algorithm 4: Pipe neighborhood change step

Procedure `Neighborhood_change_pipe`(x, x', k)
if $f(x') < f(x)$ **then**
  $x \leftarrow x'$;
else
  $k \leftarrow k + 1$;
end

from the incumbent solution. Therefore, in the so-called *skewed neighborhood change step*, a trial solution is evaluated taking into account not only the objective values of the trial and the incumbent solution but also the distance between them. The evaluation function used in the skewed neighborhood change step, in the case of the minimization of the objective function f, may be stated as:

$$g(x, y) = f(x) - f(y) - \alpha d(x, y)$$

where α represents positive parameter while $d(x, y)$ represents the distance between solutions x and y. The *skewed neighborhood change step* using this function and the sequential neighborhood change is given in Algorithm 5. Note that instead of the sequential neighborhood change, "pipe" and "cyclic" neighborhood changes may be used as well.

Algorithm 5: Skewed neighborhood change step

Procedure `Skewed_Neighborhood_change`(x, x', k, α)
if $f(x') - f(x) < \alpha d(x', x)$ **then**
  $x \leftarrow x'$;
  $k \leftarrow 1$;
else
  $k \leftarrow k + 1$;
end

2.3 Improvement procedures within VNS

2.3.1 Local search

A local search heuristic is based on the exploration of a neighborhood structure $N(x)$ of a current incumbent solution x at each iteration. Starting from an initial solution x, at each iteration it selects a better solution than x' (if any) from the predefined neighborhood structure $N(x)$ and sets it to be the new incumbent solution x. A local search heuristic finishes its work reaching an incumbent solution x such that it is already the local optimum with respect to its neighborhood structure $N(x)$. The most common search strategies used within a local search heuristic are the *first improvement* (as soon as an improving solution in a neighborhood $N(x)$ is detected, it is set to be the new incumbent solution) and the *best improvement* (the best among all improving solutions in $N(x)$ (if any) is set to be the new incumbent solution). The steps of a local search heuristic using the

first improvement strategy and a local search heuristic using the best improvement strategy are given in Algorithms 6 and 7, respectively. Note that for the sake of simplicity we assume in Algorithm 6 that $|N(x)| < \infty$. However, in the case that $|N(x)|$ is infinite or uncountable, the first improvement strategy works analogously by examining systematically solutions $x_i \in N(x)$ and making a move as soon as a an improving solution is encountered.

Note that a metaheuristic can also be used as a local search for example Simulated Annealing, Tabu Search, Iterated local search, GRASP, Genetic algorithm, etc. The Variable Neighborhood Descent described in the next section can be also used as a local search in VNS (see e.g., the description of General VNS in Section 3.4).

Algorithm 6: Local search using the first improvement search strategy.

```
Function LS_FI(x, N)
repeat
    Let N(x) = {x^1, ..., x^p};
    i ← 0;
    x' ← x;
    repeat
        i ← i + 1;
        if f(x^i) < f(x) then
            x ← x^i;
            break;
        end
    until i = p;
until f(x') ≤ f(x);
return x';
```

Algorithm 7: Local search using the best improvement search strategy.

```
Function LS_BI(x, N)
repeat
    x' ← x;
    x ← argmin_{y ∈ N(x')} f(y);
until f(x') ≤ f(x);
return x';
```

2.3.2 Variable neighborhood descent procedures

The Variable Neighborhood Descent (VND) procedures exploit the fact that the solution which is a local optimum with respect to several neighborhood structures is more likely to be a global optimum than the solution generated as a local optimum for just one neighborhood structure. More precisely a VND procedure explores several neighborhood structures either in a sequential or nested fashion in order to possible improve a given solution. As a search strategy it may use either the first improvement or the best improvement search strategy. One example of

VND procedure is the improvement procedure of Pivot and Complement heuristic, a heuristic for solving a pure 0-1 Mixed Integer Programs (MIP) [4].

2.3.2.1 Sequential variable neighborhood descent procedures

The basic sequential VND (B-VND) procedure works in the following way. Several neighborhood structures are firstly ordered in a list and after that examined one after another respecting the established order. Let $N = \{N_1, \ldots, N_{\ell_{max}}\}$ be a set of operators defining the neighborhoods structures and the order of their examination. Starting from a given solution x, the basic sequential VND procedure iteratively explores its neighborhood structures defined by the operators N_ℓ, $1 \leq \ell \leq \ell_{max}$ one after another according to the established order. As soon as an improvement of the incumbent solution in some neighborhood structure occurs, the basic sequential VND procedure resumes search in the first neighborhood structure (according to the defined order) of the new incumbent solution. The whole process is stopped if the current incumbent solution can not be improved with respect to any of ℓ_{max} neighborhood structures. The steps of the sequential VND using the best improvement search strategy are given in Algorithm 8. Besides this basic sequential VND procedure, two sequential VND procedures have been proposed using the different rules for selecting the next neighborhood structure to be explored if an improvement of current incumbent solution occurs:

- **pipe VND (P-VND)** : uses the pipe neighborhood change step to decide which neighborhood will be explored as the next. Thus, the steps of pipe VND using the best improvement search strategy may be deduced from the Algorithm 8, replacing the neighborhood change procedure `Neighborhood_change_sequential(`x`,` x'`,` ℓ`)` given in Algorithm 2 by the procedure `Neighborhood_change_pipe(`x`,` x'`,` ℓ`)` given in Algorithm 4. Note that the P-VND sometimes is referred to as the token ring search (see e.g., [14, 44]).
- **cyclic VND (C-VND)** : uses the cyclic neighborhood change step to decide which neighborhood will be explored the next. Thus, the steps of cyclic VND using the best improvement search strategy may be deduced from Algorithm 8, replacing the neighborhood change procedure `Neighborhood_change_sequential` given in Algorithm 2 by the procedure `Neighborhood_change_cyclic(`x`,` x'`,` ℓ`)` given in Algorithm 3. The approach presented in [24] could be considered as a cyclic VND except that it has cycles within cycles and uses additional evaluation criteria along the way.

Finally, the last sequential VND variant is **Union VND (U-VND)** (sometimes called Multiple neighborhood search) which at each iteration explores the single neighborhood, obtained as the union of all predefined ℓ_{max} neighborhoods, trying to improve the current incumbent solution. If the union does not contain any improving solution, then U-VND finishes its work; otherwise, the best among improving solutions is set to be new incumbent solution and the process is resumed. U-VND is usually used within Tabu search (see e.g., [44, 68]).

Note that each of these VND algorithms may also use the first improvement search strategy to explore neighborhood structures. If the first improvement strategy is used, the steps of each VND variant are the same as the steps of corresponding VND variant that uses best improvement strategy but with additional

assumption that the call $\text{argmin}_{y \in N_\ell(x)} f(y)$ returns the first encountered solution y in the neighborhood $N_\ell(x)$ such that $f(y) < f(x)$.

Algorithm 8: Sequential VND using the best improvement search strategy.

```
Function B-VND(x, ℓ_max, N)
repeat
    stop ← false;
    ℓ ← 1;
    x' ← x;
    repeat
        x'' ← argmin_{y∈N_ℓ(x)} f(y);
        Neighboorhood_change_sequential(x, x'', ℓ);
    until ℓ = ℓ_max;
    if f(x') ≤ f(x) then
        stop ← true;
    end
until stop = true;
retourner x';
```

Empirical study. In [45] an empirical study on performances of VND variants used to solve the traveling salesman problem (TSP) is performed. For testing purposes, random test instances were generated in the way described in [28]. Within VND variants, three classical TSP neighborhood structures are considered: 2-opt (Figure 1), Insertion-1 (Figure 2) and Insertion-2 (Figure 3). On each instance from this data set, each VND variant, except U-VND, is tested under 24 different settings. Each setting corresponds to choosing the following: (i) one out of two common ways for getting an initial solution: at *random* (solution generated as a random permutation of nodes) or *greedy*; (ii) one out of six possible neighborhood orders, and (iii) the *best* or the *first improvement search strategy*. This gives $2 \times 6 \times 2 = 24$ different search methods that use 2-opt, Insertion-1 and Insertion-2 neighborhoods. On the other hand, U-VND is tested under only two different settings as: U-VND that uses the best improvement search strategy and the greedy initial solution and U-VND that uses the best improvement search strategy and the random initial solution. Note that if the first improvement search strategy is used within U-VND, then U-VND is equivalent to B-VND.

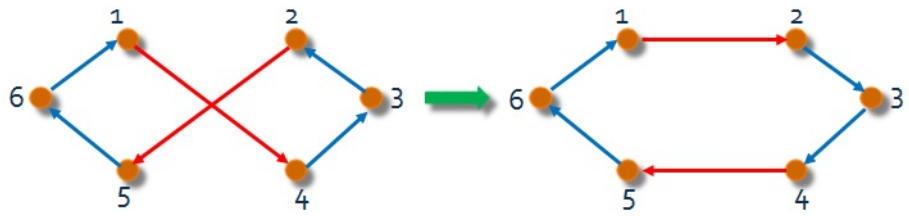


Fig. 1: 2-opt move

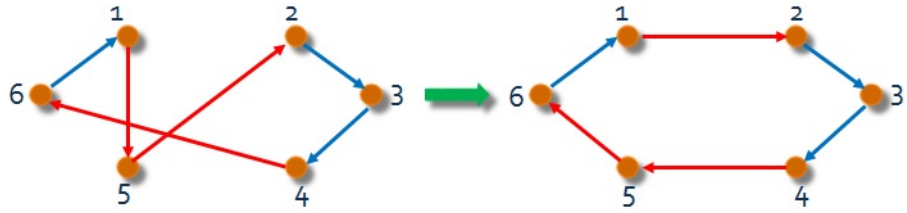


Fig. 2: Insertion-1 move

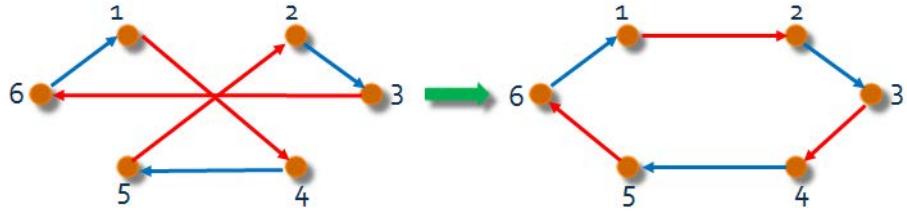


Fig. 3: Insertion-2 move

Table 1 summarizes results considering the entire set of 15,200 instances as a test case. Namely, the average solution values (Column '`av. best value`') and average CPU times (Column '`av. time`') over all test instances in the data set, attained by VND variants under the best settings are reported.

VND variant	av. best value	av. time (s)
Basic VND	1198.24	0.16
Pipe VND	1198.52	0.12
Cyclic VND	1198.76	0.46
Union VND	1197.65	1.06

Table 1: Comparison of VND variants

From the results presented in Table 1 following conclusions may be drawn:

(i) U-VND is slightly better than the others VNDs, regarding the best average values attained, but much slower than the others. This is explained by the fact that U-VND in each iteration performs the exploration of large part of the solution space before deciding to re-center the search. Obviously, such principle is suitable for reaching good final solution, but requires a large CPU time.

(ii) Comparing VNDs that re-center the search in the inner loop (i.e., B-VND, P-VND and C-VND), it follows that B-VND is able to provide the best solution values. Regarding average CPU times consumed by B-VND, P-VND and

C-VND to find the best reported average solution value, their ranking is as follows: the fastest is P-VND, B-VND is ranked as the second, while C-VND is the slowest one. However, B-VND consumed negligible more CPU time to find the best reported average solution value, comparing to CPU time P-VND consumed, we may conclude that B-VND is the most appropriate VND version among those that re-center search in the inner loop.

2.3.2.2 Nested variable neighborhood descent procedures

A nested (composite) variable neighborhood descent procedure [32] explores a large neighborhood structure obtained as a composition of several neighborhoods. More precisely, let $N = \{N_1, \dots N_{\ell_{max}}\}$ again be set of operators such that each operator N_ℓ, $1 \leq \ell \leq \ell_{max}$ maps a given solution x to a predefined neighborhood structure $N_\ell(x)$. Then, the neighborhood explored within a nested variable neighborhood procedure is defined by operator $N^* = N_1 \circ N_2 \circ \cdots \circ N_{\ell_{max}}$. The cardinality of a neighborhood structure $N^*(x) = N_1(N_2(\dots(N_{\ell_{max}}(x))))$ of some solution x is bounded by $\prod_{\ell=1}^{\ell_{max}} n_\ell$ where $n_\ell = max\{|N_\ell(x)| : x \in X\}$, $1 \leq \ell \leq \ell_{max}$.

Such a large cardinality obviously increases chances to find an improvement in the neighborhood. The steps of nested VND using the best improvement search strategy are given in Algorithm 9. The neighborhood $N^*(x)$ may be also explored using the first improvement search strategy, following steps of local search procedure using the first improvement strategy given in Algorithm 6. However, since its cardinality is usually very large, the first improvement is used more often [32, 66].

Algorithm 9: Nested VND using the best improvement search strategy.

Function `Nested_VND`(x, ℓ_{max}, N)
$N^* = N_1 \circ N_2 \circ \cdots \circ N_{\ell_{max}}$
repeat
  $x' \leftarrow x$;
  $x \leftarrow argmin_{y \in N^*(x')} f(y)$;
until $f(x') \leq f(x)$;
return x';

2.3.2.3 Mixed variable neighborhood descent procedures

Mixed variable neighborhood descent (mixed VND) [32] combines ideas of sequential and nested variable neighborhood descent. Namely, it uses a set of operators $N' = \{N_1, \dots N_b\}$ to define a nested neighborhood and each time a solution in this nested neighborhoods is visited a sequential variable neighborhood descent variant defined by a set of operators $N'' = \{N_{b+1}, \dots N_{\ell_{max}}\}$ is launched. The cardinality of the set explored in one iteration of a mixed VND is bounded by

$$|N_{mixed}(x)| \leq \prod_{\ell=1}^{b} n_\ell \sum_{\ell=b+1}^{\ell_{max}} n_\ell, x \in X$$

where $n_\ell = max\{|N_\ell(x)| : x \in X\}$, $1 \leq \ell \leq \ell_{max}$.

Note that if the set N' is the empty set (i.e., $b = 0$) we get pure sequential VND. If $b = \ell_{max}$ we get pure nested VND. Since nested VND intensifies the

search in a deterministic way, boost parameter b may be seen as a balance between intensification and diversification in deterministic local search with several neighborhoods.

In Algorithm 10 we give steps of a mixed VND where the basic sequential VND presented in Algorithm 8 is applied on each point of a nested neighborhood. The best improvement search strategy is used within both the nested neighborhood and the basic sequential VND. Note that mixed VND may be implemented using the first improvement search strategy either within a nested neighborhood or a sequential VND as well as using a pipe or a cyclic VND instead of a basic sequential VND. The ideas similar to those of mixed VND have been exploited in the filter&fan and the ejection chains methods, see [34, 57] and the references therein.

Algorithm 10: Mixed VND using the best improvement search strategy.

Function `Mixed_VND`(x, b, ℓ_{max}, N', N'')
$N^* = N_1 \circ N_2 \circ \cdots \circ N_b$
$x' \leftarrow x$;
repeat
  $stop = true$;
  $x \leftarrow x'$;
  for *each* $y \in N^*(x)$ **do**
    $x'' \leftarrow$ `B-VND`(y, $\ell_{max} - b$, N'') ;
    if $f(x'') < f(x')$ **then**
      $stop = false$
      $x' \leftarrow x''$;
    end
  end
until $stop = true$;
return x';

In Table 2, we present which VND variants and which neighborhood change steps are used together within VNS framework up to now. Sign '`+`' in the table means that there is a VNS variant that employs together certain VND variant and neighborhood change procedure, while sign '`-`' stands for the opposite. Note that, in Table 2 the neighborhood change step provided under column '`neigh. change step`' refers to the one used within a VNS not the one used within the VND. From the table it follows that the sequential neighborhood change step is the one widely used. Recently, the sequential neighborhood change step is used together with: (i) B-VND in [1, 11, 36, 50]; (ii) P-VND in [64, 65]; (iii) C-VND in [63]; (iv) Nested_VND in [66]; (v) Mixed_VND in [32]. On the other hand, the skewed neighborhood change step has been used in combination with B-VND in [8, 52]. To the best of our knowledge, the other combinations of neighborhood change steps and VND procedures have not been investigated yet and represent an open chapter for the future research in the area of optimization.

	VND variant					
neigh. change step	B-VND	P-VND	C-VND	U-VND	Nested_VND	Mixed_VND
sequential	+	+	+	-	+	+
pipe	-	-	-	-	-	-
cyclic	-	-	-	-	-	-
skewed	+	-	-	-	-	-

Table 2: Existing combinations of VND variants and neighborhood change step procedures

3 Variable neighborhood search variants

3.1 Fixed neighborhood search

Fixed neighborhood search (FNS) [7], also known as Iterated Local Search (ILS) [33], is a step in between classical local search and variable metric on the one hand and VNS on the other hand. Instead of generating initial solutions completely at random, the next starting point for local search in FNS is a randomly generated solution taken from the vicinity of the best one found so far (incumbent solution). With this simple modification of the multi-start local search (MLS), two advantages are obtained: (i) improved effectiveness: some of the solution attributes with good values in the incumbent are kept; (ii) improved efficiency: the next local search will have fewer iterations, since the incumbent will be in a deeper valley of the solution space. In order to find a perturbed solution, one needs to define a neighborhood structure $\mathcal{N}(x)$ different from $N(x)$, the one used in the local search. The perturbed solution x' will belong to $\mathcal{N}(x)$. The FNS (or ILS) procedure is given in Algorithm 11.

Algorithm 11: Fixed neighborhood search

```
   Function FNS (x, 𝒩, N, t_max)
1  f_best ← ∞
2  repeat
3  |  Generate perturb point x' ∈ 𝒩(x) at random;
4  |  x' ← LocalSearch(x', N)
5  |  if f(x') < f_best then
6  |  |  x ← x'; f_best = f(x);
   |  end
7  |  t ← CpuTime()
   until t > t_max
8  return x;
```

3.2 Basic VNS

The basic VNS variant executes alternately a simple local search procedure (one of two presented in Algorithms 6 and 7) and a shaking procedure presented in Algorithm 1 together with a neighborhood change step (one of these presented in Algorithms 2-5) until fulfilling a predefined stopping criterion. As a stopping

criterion, most often, is used maximum CPU time allowed to be consumed by Basic VNS. The steps of Basic VNS using sequential neighborhood change step are given in Algorithm 12.

Algorithm 12: Basic Variable Neighborhood Search.

Function `Basic_VNS`(x, k_{max}, $\mathcal{N}$,N)
repeat
  $k \leftarrow 1$;
  while $k \leq k_{max}$ **do**
    $x' \leftarrow$ `Shake`$(x, k, \mathcal{N})$;
    $x'' \leftarrow$ `Local_search`(x', N);
    `Neighborhood_change_sequential`(x, x'', k);
  end
until *stopping condition is fulfilled;*
return x;

3.3 Reduced VNS

From this basic VNS scheme several other VNS approaches have been derived. The simplest one is so-called *Reduced VNS*, which employs a shaking procedure and a neighborhood change step procedure while the improvement phase is discarded. The steps of *Reduced VNS* that uses sequential neighborhood change function are given in Algorithm 13. Note that Monte Carlo Method is a special case of Reduced VNS (i.e., if $k_{max} = 1$ and $\mathcal{N}_1(x) = X$ then Reduced VNS turns out to be Monte Carlo Method).

Algorithm 13: Reduced Variable Neighborhood Search.

Function `Reduced_VNS`(x, k_{max}, $\mathcal{N}$)
repeat
  $k \leftarrow 1$;
  while $k \leq k_{max}$ **do**
    $x' \leftarrow$ `Shake`$(x, k, \mathcal{N})$;
    `Neighborhood_change_sequential`(x, x', k);
  end
until *stopping condition is fulfilled;*
return x;

3.4 General VNS

Another variant is so-called *General VNS* which as an improvement procedure uses some of VND procedure presented above unlike to Basic VNS which uses just a simple local search. Note that it is not obligatory that neighborhood structures used within the shaking procedure and a VND procedure are the same although it is more desirable. The steps of a General VNS that uses sequential neighborhood

change function are given in Algorithm 14. In the algorithm the statement of the form `VND`(x', ℓ_{max}, N) means that a certain VND variant presented above is executed.

Algorithm 14: General Variable Neighborhood Search.

Function `General_VNS`(x, k_{max}, ℓ_{max}, $\mathcal{N}$, N)
repeat
  $k \leftarrow 1$;
  while $k \leq k_{max}$ **do**
    $x' \leftarrow$ `Shake`$(x, k, \mathcal{N})$;
    $x'' \leftarrow$ `VND`(x', ℓ_{max}, N);
    `Neighborhood_change_sequential`(x, x'', k);
  end
until *stopping condition is fulfilled;*
return x;

As previously shown, regarding the best average solution quality, the best VND approach is U-VND, while the best VND approach among those that re-center search in the inner loop is B-VND. However, U-VND is significantly slower than B-VND. Thus, in [45] series of experiments are conducted in order to reveal whether is more beneficial to use within VNS, a VND that re-centers search in the inner loop or not. For that purpose, two GVNS heuristics are implemented and tested. The first one named `GVNS_B-VND` uses B-VND as a local search, while the second named `GVNS_U-VND` employs U-VND as a local search. U-VND and B-VND used within tested GVNS heuristics explore again neighborhood structures 2-opt, Insertion-1 and Insertion-2. The search strategy and order of neighborhoods used within B-VND are determined as the ones that enable B-VND to achieve the highest performance. The shaking procedure used, for the input requires solution x and parameter k, while at the output it returns a solution obtained executing k random 2-opt moves on the solution x. Following the observation that the best initial solution for all VND approaches is the one built by a greedy procedure, we provide the same greedy solution as the initial one for both GVNS heuristics. The performances of `GVNS_B-VND` and `GVNS_U-VND` have been disclosed on 23 TSP instances from TSPLIB [58]. On each test instance, each GVNS has been executed ten times with different random seeds. In addition, we test both GVNS using 12 different time limits ranging from 10 seconds to 300 seconds. In all testing GVNS parameter k_{max} is set to 10. The results are summarized in figure 4.

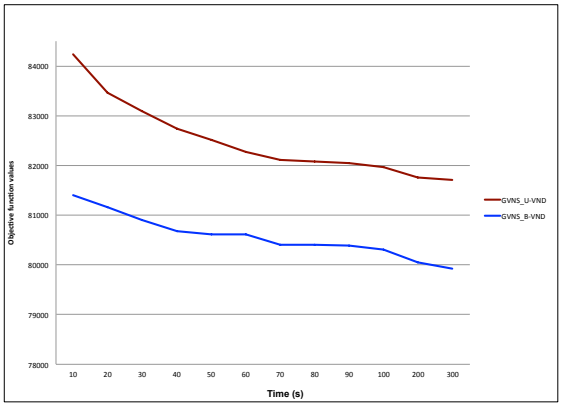


Fig. 4: GVNS_U-VND versus GVNS_B-VND

From the results presented in Figure 4, we may draw the following conclusions. `GVNS_B-VND` significantly outperforms `GVNS_U-VND` regarding solution quality for each time limit imposed. In addition, it is worth mentioning that even with the time limit of 300 seconds `GVNS_U-VND` cannot reach neither the best nor the average solution quality obtained by `GVNS_B-VND` after 10 seconds of execution.

All in all, these observations, undoubtable confirm superiority of `GVNS_B-VND` over `GVNS_U-VND`. Such outcome may be explained by the fact that U-VND used within `GVNS_U-VND` is very slow compared to B-VND as demonstrated in the previous subsection. In addition, since at each iteration U-VND generates local optimum with respect to three neighborhood structures, it may get stuck in some local optima valley with higher probability than a VND variant that re-center the search in the inner loop. Thus, we may conclude that in order to achieve high performance of GVNS, it is preferable to use VND variant that in all iterations, but the last, re-center the search around a new solution which is not a local optimum for the union of neighborhoods.

3.5 Skewed VNS

Skewed VNS (S-VNS) variants are those that in the neighborhood change step accept as new incumbent solutions not only improving solutions but in some cases those which are worse than the current incumbent solution. Therefore, in the so-called *skewed neighborhood change step*, a trial solution is evaluated taking into account not only the objective values of the trial and the incumbent solution but also the distance between them as shown in Algorithm 5. In that way Skewed VNS may successfully resolve possible local optima traps exploring valleys far from the incumbent solution.

3.6 Nested VNS

One generalization of VNS called *nested VNS* [61] consists in applying a VNS variant on each point of a predefined nested (composed) neighborhood structure, instead of applying a local search or a VND variant as it is usual. Hence, it may be also seen as a generalization of mixed VND (see Algorithm 10). In Algorithm 15 we give steps of nested VNS which uses the best improvement search strategy to explore the given neighborhood structure (although the first improvement search strategy may be used as well). For performance of *nested VNS* it is crucial that the VNS applied at each point of the predefined neighborhood is very fast. For that reason it is desirable to use small CPU time limit as a stooping criterion for this VNS.

Algorithm 15: Nested VNS using the best improvement search strategy.

```
Function Nested_VNS(x, b,k_max, ℓ_max, 𝒩, N', N'' )
N* = N'_1 ∘ N'_2 ∘ ··· ∘ N'_b;
x' ← x;
repeat
    stop = true;
    x ← x';
    for each y ∈ N*(x) do
        x'' ← VNS(y, k_max, ℓ_max, 𝒩, N'') ;
        if f(x'') < f(x') then
            stop = false;
            x' ← x'';
        end
    end
until stop = true;
return x';
```

Since the Nested VNS iterates until there is no improvement, it may be seen as a local search procedure. Therefore, it may be used as the improvement procedure of a VNS heuristic. In that case, the resulting VNS is called **two-level VNS** [51].

4 Advanced VNS variants

4.1 Variable neighborhood decomposition search

The main idea of *the variable neighborhood decomposition search* (VNDS) [30] is to systematically change the size of subproblems that appear in the decomposition (from smaller to larger). Moreover the subproblems obtained by decomposition are solved by VNS, yielding a two-level VNS approach. Namely, unlike other VNS variants VNDS does not launch an improvement phase in the whole solution space of a considered problem, but within a reduced solution space that corresponds to a reduced problem derived from the original one. The steps of VNDS are given in Algorithm 16. In this algorithm, the solution y corresponds to the solution obtained by decomposing the problem. Usually, it is generated so that it has k attributes which are different from those in the current incumbent solution x. On such generated solution an improvement phase is applied in the reduced solution

space. Then, the solution returned by the improvement procedure is used to create the solution of the original problem. Namely, the solution returned by the improvement procedure together with the partial solution $x' \setminus y$ constitutes a solution of the original problem. As an improvement procedure within VNDS may be used some local search procedure, VND variant or some VNS variant.

Algorithm 16: Variable Neighborhood Decomposition Search

Function VNDS(x, k_{max}, $\mathcal{N}$)
repeat
  $k \leftarrow 1$;
  repeat
    $x' \leftarrow$ `Shake`$(x, k, \mathcal{N})$;
    $y \leftarrow x' \setminus x$;
    $y' \leftarrow$ `Improvement_procedure`(y);
    $x'' = (x' \setminus y) \cup y'$;
    `Neighborhood_change_sequential`(x, x'', k);
  until $k = k_{max}$;
until *stopping condition is fulfilled;*
return x;

4.2 Primal-dual VNS

The primal-dual VNS [27] is a variant of VNS that provides the estimation of quality of the obtained solution unlike the other VNS variants. Namely, the primal-dual VNS in the first phase uses a VNS heuristic to obtain a primal feasible solution. After that this solution is used to deduce a dual (infeasible) solution. On a such obtained dual solution, a VNS heuristic is applied in order to reduce dual infeasibility followed by an exact method to solve relaxed dual problem and generate a lower bound. Finally, a standard branch-and-bound algorithm is launched to find an optimal solution of the original problem using tight upper and lower bounds, obtained from the heuristic primal solution and the exact dual solution, respectively.

4.3 VNS for nonlinear optimization

We assume that $f : \mathbb{R}^n \to \mathbb{R}$ is a continuous function in (1). No further assumptions are made on f. In particular, f does not need to be convex or smooth.

4.3.1 Glob-VNS

Several VNS based methods for solving continuous (un)constrained optimization problems have been proposed in the literature. For example, radar polyphase code design is considered in [48], [3] solves the bilinear (pooling) problem, while [15, 41] considers Distance Geometry. Drazić et al. [17] suggests a VNS based heuristic called `Glob-VNS` for solving general unconstrained optimization problems. The pseudo-code for `Glob-VNS` is given in Algorithm 17 below.

Algorithm 17: `Glob-VNS` algorithm for unconstrained optimization

Function `Glob-VNS` (x, k_{max}, t_{max})
1 Select the set of neighborhood structures $\mathcal{N}_k$, $k = 1, \ldots, k_{\max}$
2 Select the array of distribution types $\mathcal{D}_j, j = 1, \ldots, r$
3 $t \leftarrow 0$
4 **while** $t < t_{max}$ **do**
5 $k \leftarrow 1$ // Initialize neighborhood counter k
6 **repeat**
7 **for** *all distribution types* $\mathcal{D}_j$ **do**
8 $x' \leftarrow$ `Shake`(x, k) // Get $x' \in \mathcal{N}_k(x)$ at random with $\mathcal{D}_j$ distribution
9 $x'' \leftarrow$ `LocalSearch`(x') // Apply Local Search to get a local minimum x''
10 **if** $f(x'') < f(x)$ **then**
11 $x \leftarrow x''$, `go to 5`
 end
 end
12 $k \leftarrow k + 1$
 until $k = k_{\max}$
13 $t \leftarrow$ `CpuTime`()
 end
14 **return** x;

Beside t_{max} and k_{max}, the choice of the local search solver may be seen as an additional parameter of `Glob-VNS`. The user can use one out of six well-known unconstrained methods offered in the software described in [17]. Three are first order methods (Steepest descent, Fletcher-Reeves, Variable metric) while the other three are direct search methods (Nelder-Mead, Hooke-Jeeves, Rosenbrock). In the Shaking step there are two aspects that need to be considered. One is the set of neighborhood structures $\mathcal{N}_k$ (induced from some metric such as the ℓ_p norm), while the other concerns how to take a random point from $\mathcal{N}_k(x)$. This second question relates to the distribution $\mathcal{D}_j$ used to select the point. Neighborhood structures $\mathcal{N}_k$ are usually induced from the ℓ_∞ norm. Radii around the incumbent are found automatically, taking into account the box constraints: each coordinate is divided in k_{max} intervals from both sides of the incumbent, forming k_{max} concentric hypercubes.

4.3.2 Gauss-VNS

VNS variants that do not use metric distances to define neighborhoods are proposed in [67]; [46] extends the unconstrained solver `GLOB-VNS` to the constrained case, using an exterior penalty function method; [2] and [6] hybridize VNS with Trust region methods; in [10] `GLOB-VNS` is modified by the so called Gaussian VNS (`Gauss-VNS`), whose steps are given below (Algorithm 18).

The paradigm of neighborhoods is generalized in the Gauss VNS approach so that problems with unbounded domains can be addressed. The idea is to replace the class $\{\mathcal{N}_k(x)\}_{1 \le k \le k_{\max}}$ of neighborhoods of point x by a class of *probability distributions* $\{\mathcal{P}_k(x)\}_{1 \le k \le k_{\max}}$. The next random point in the shaking step is generated using the probability distribution $\mathcal{P}_k(x)$. The probabilistic analysis linked to the choice of distribution could be an interesting idea for the future work.

Algorithm 18: `Gauss-VNS` algorithm for unconstrained optimization

Function `Gauss-VNS` (x, k_{max}, t_{max})
1 Select the set of covariance matrices Σ_k, $k = 1, \ldots, k_{\max}$
2 $t \leftarrow 0$
3 **while** $t < t_{max}$ **do**
4 $k \leftarrow 1$
5 **repeat**
6 $x' \leftarrow$ `Shake`(x, k) // Get x' from a Gaussian distribution with // mean x and covariance matrix Σ_k
7 $x'' \leftarrow$ `LocalSearch`(x') // Apply Local Search to get a local minimum x''
8 $k \leftarrow k + 1$
9 **if** $f(x'') < f(x)$ **then**
10 $x \leftarrow x''$, $k \leftarrow 1$
  end
  until $k = k_{\max}$
11 $t \leftarrow$ `CpuTime`()
  end
12 **return** x;

4.3.3 VNS with Modified Nelder Mead as a local search

A weak point of both `Glob-VNS` and `Gauss-VNS` is the fact that the choice of a local search procedure is left completely to the user (steps 9 and 7 in `Glob-VNS` and `Gauss-VNS`, respectively). In order to make the VNS method more user friendly, a restarted modified Nelder-Mead (NM) method [70, 71] is suggested in [16] as the local search for any type of optimization problem. The basic ideas of restarted modified NM (RMNM) are: (i) a shrink move is performed one vertex at a time, instead of moving all vertices to the direction of the best; (ii) parameter values $\alpha, \beta, \gamma, \delta$ are taken at random from a given interval, i.e., the next simplex vertex is obtained by

$$x_{new} = (1 + g) \cdot \bar{x} - g \cdot x_{n+1}, \tag{2}$$

where $\bar{x}$ is the centroid of the simplex $X = \{x_1, \ldots, x_n, x_{n+1}\} \setminus \{x_{n+1}\}$, x_{n+1} is a simplex vertex with the worst objective function value, and $g = \alpha, \alpha \cdot \beta, \alpha \cdot \gamma$ or $-\gamma$, expressing $reflection$, *expansion*, *inside contraction* and *outside contraction*, respectively [70]; (iii) once the volume of the simplex is less than an arbitrarily small number ϵ, the procedure is restarted with the initial value of the simplex side length *size* unless no improvement has been reached from the previous big iteration. The steps of the RMNM used in the local search routine of `Glob-VNS` are given in Algorithm 19.

In step 2 the initial simplex $X = \{x_1, \ldots, x_{n+1}\}$ is generated around a given point $x_1 \in \mathbb{R}^n$, and then its vertices are ordered according to their quality ($f(x_1) \leq \cdots \leq f(x_{n+1})$). If a Nelder-Mead iteration is unsuccessful, i.e., if there is no point on the line segment that connects x_{n+1} and centroid $\bar{x}$ with better objective function value, we perform the shrink step only for one vertex x_j of the simplex X. Otherwise, the simplex is updated: the worst point is swapped with the new one, and all vertices of X are again ranked according to their objective function values. Once a local minimum is reached (a big iteration ends), the procedure is restarted with the same simplex size as the initial one, but now centered around the new local minimum. This restart procedure allows RMNM to try to escape from the current local minimum.

Algorithm 19: Restarted Modified NM (RMNM)

```
   function RMNM(f, n, x*, size)
1  x_1 ← InitialPoint; j ← 0
   repeat
2  |  X ← InitialSimplex(x_1, size); X* ← X; x* ← x_1
3  |  while Stopping condition is not met do
4  |  |  if not MNMIteration(X) then
5  |  |  |  X ← X*; j ← j + 1
6  |  |  |  X ← Shrink(X, j)
7  |  |  |  if j = n then
8  |  |  |  |  X* ← X; j ← 0
   |  |  |  end
   |  |  else
9  |  |  |  X* ← X; j ← 0
   |  |  end
   |  end
   until f(x_1) < f(x*)
10 return x*
```

4.4 VNS for Mixed Integer Non-Linear Programs

In recent papers [42, 43] an effective and reliable Mixed Integer Non-Linear Programming (MINLP) heuristic based on VNS is suggested, called Relaxed-Exact Continuous-Integer Problem Exploration (RECIPE for short). RECIPE puts together a global search phase based on VNS and a local search phase based on a Branch and Bound (B&B) type heuristic. The VNS global phase relies on neighborhoods defined as hyperrectangles for the continuous and general integer variables and by local branching constraints for the binary variables. The local phase employs a B&B solver for convex MINLPs [21], which is applied to non convex MINLPs heuristically. A local solution using a Sequential Quadratic Programming (SQP) algorithm [23] supplies an initial constraint-feasible solution to be employed by the B&B as initial upper bound. RECIPE (see Algorithm 20) is an efficient, effective and reliable general-purpose algorithm for solving complex MINLPs of small and medium scale. Note that beside t_{max} and k_{max}, the input value in Algorithm 20 is the initial solution x^*. The original contribution of RECIPE is the particular combination of some well-known and well-tested tools to produce a very powerful global optimization method. It turns out that RECIPE, acting on the whole MINLPLib library [9], is able to find optima equal to or better than those reported in the literature for 55% of the instances. The closest competitor is SBB+CONOPT with 37%. The known optima are improved in 7% of the cases.

4.5 Variable formulation space search

It is well known that many optimization problems have more than one formulation. In the case that a problem has several formulations very often it appears that a local optimum with respect to one formulation is not necessarily a local optimum with respect to another formulation. This fact has been exploited by *Variable formulation space search* [49]. Its main idea is to use several formulations of a considered problem, and to switch from one to another formulation, depending on

Algorithm 20: The RECIPE heuristic for solving MINLP

```
Function RECIPE (k_max, t_max, x*)
t ← 0
while t < t_max do
    k ← 1
    while k ≤ k_max do
        i ← 1
        while i ≤ b do
            Sample x̄ ∈ N_k(x*) at random
            x ← SQP(x̄)
            if x not feasible then
                x ← x̄
            end
            x' ← B&B (x, k, k_max)
            if x' is better than x* then
                x* ← x'; k ← 0; break;
            end
            i ← i + 1
        end
        k ← k + 1
    end
    t ← CpuTime()
end
return x*;
```

the current state of the solution process. When the change from one formulation to another occurs, the solution resulting from the former formulation is used as an initial solution for the latter formulation. In Algorithm 21, a formulation change procedure is given. In this procedure, notation $f(\phi, x)$ corresponds to the objective value of the solution x in the formulation ϕ, while assignment $\phi \leftarrow \phi'$ represents the change of a formulation.

One variant of Variable formulation space search is so called *Variable Formulation Search*. It uses alternative formulations of the problem to determine which solution is more promising when they have the same value of the objective function in the original formulation. Variable formulation search was applied to the Cutwidth Minimization Problem in [54].

Algorithm 21: Formulation change procedure

```
Procedure Formulation_change(x, x', φ, φ',k)
if f(φ', x') < f(φ, x) then
    x ← x'; φ ← φ'; k ← 1;
else
    k ← k + 1;
end
```

4.6 Matheuristics with VNS

Variable neighborhood search has been combined with solvers for mixed integer programs (MIP) in the context of providing a first feasible solution as well as finding good quality solutions. Here, we present such heuristics used to tackle 0-1 mixed integer programs. A 0–1 mixed integer program (MIP) may be written in the following form:

$$(MIP)\begin{cases}\min\ v = cx \\ s.t.\ Ax \le b \\ 0 \le x_j \le U_j,\ j \in V = \{1,\dots,n\} \\ x_j \in \{0,1\},\ j \in \mathcal{I} \subseteq V\end{cases} \tag{3}$$

where A is a $m \times n$ constant matrix, b is a constant vector, the set V denotes the index set of variables, while the set $\mathcal{I}$ contains indices of binary variables. Each variable x_j has an upper bound denoted by U_j (which equals 1 if x_j is binary variable, while otherwise may be infinite). The integer problem defined in this manner will be denoted simply by MIP while the relaxation of MIP obtained by excluding integrality constraints will be denoted by LP. A feasible solution of $MIP(LP)$ will be called $MIP(LP)$ feasible. An optimal solution of the LP problem will be denoted by $\overline{x}$. The set of all MIP feasible solutions will be denoted by X, i.e., $X = \{x \in \mathbb{R}^n : Ax \le b;\ 0 \le x_j \le U_j,\ j \in V = \{1,\dots,n\};\ x_j \in \{0,1\},\ j \in \mathcal{I} \subseteq V\}$. An optimal solution or the best found solution obtained in an attempt to solve the MIP problem will be denoted by x^*, while its objective value will be denoted by v^*.

Hamming distance between two solutions x and x' such that $x_j, x'_j \in \{0,1\},\ j \in \mathcal{I}$ is defined by

$$\delta(x,x') = \sum_{j\in\mathcal{I}} |x_j - x'_j| = \sum_{j\in\mathcal{I}} x_j(1-x'_j) + x'_j(1-x_j).$$

The partial Hamming distance between x and x', relative to the subset $J \subset \mathcal{I}$, is defined as $\delta(J,x,x') = \sum_{j\in J} \mid x_j - x'_j \mid$ (obviously, $\delta(\mathcal{I},x,x') = \delta(x,x')$).

The MIP relaxation of the 0-1 MIP problem relative to a subset $J \subset \mathcal{I}$ is expressed as:

$$(MIP(J))\begin{cases}\min\ v = cx \\ s.t.\ Ax \le b \\ 0 \le x_j \le U_j,\ j \in V = \{1,\dots,n\} \\ x_j \in \{0,1\},\ j \in J \subset \mathcal{I} \subseteq V\end{cases} \tag{4}$$

Given a MIP problem, $P\ \min\{cx \mid x \in X\}$ and an arbitrary LP feasible solution x^0. The problem *reduced* from the original problem P and associated with x^0 and a subset $J \subseteq \mathcal{I}$ is defined as:

$$P(x^0, J)\ \min\{cx \mid x \in X, x_j = x_j^0 \text{ for } j \in J \text{ such that } x_j^0 \in \{0,1\}\} \tag{5}$$

Note that in the case that $J = \mathcal{I}$, the reduced problem will be denoted by $P(x^0)$.

Similarly, given a MIP problem P and a solution $\widetilde{x}$ such that $\widetilde{x}_j \in \{0,1\},\ j \in \mathcal{I}$. Then, $MIP(P,\widetilde{x})$ will denote a minimization problem, obtained from MIP problem P by replacing the original objective function with $\delta(x,\widetilde{x})$:

$$MIP(P,\tilde{x}) \qquad \min\{\delta(\tilde{x},x) \mid x \in X\}. \tag{6}$$

The LP relaxation of such defined MIP problem $MIP(P, \tilde{x})$ will be denoted by $LP(P, \tilde{x})$.

If C is a set of constraints, we will denote with $(P \mid C)$ the problem obtained by adding all constraints in C to the problem P.

Let α be a real number, then $near(\alpha)$ will refer to the nearest integer value of a real value α i.e., $near(\alpha) = \lfloor \alpha + 0.5 \rfloor$, where $\lfloor \alpha + 0.5 \rfloor$ represents the integer part of the number $\alpha + 0.5$ (i.e., the greatest integer $\leq \alpha + 0.5$). Furthermore, let x be a vector, then $near(x)$ will represent the nearest integer vector relative to the vector x, by defining each component as $near(x)_j = near(x_j) = \lfloor x_j + 0.5 \rfloor$.

4.6.1 Variable Neighborhood Branching

In 2006, Hansen et al. [31] proposed variable neighborhood search (VNS) heuristic combined with Local Branching (LB) [20], called *Variable Neighborhood Branching* (see Algorithm 22), for solving mixed-integer programs which may be seen as a generalization of Local Branching. The main advantage of the proposed VNS comparing to the LB heuristic is the fact that it performs more systematic neighborhood exploration than Local Branching [20].

Algorithm 22: Variable Neighborhood Descent branching

```
    Function VND-MIP(P, t_vnd, t_mip, r_max, x');
 1  Set r = 1, t_start = CpuTime(), t = 0;
 2  Set Q = P;
 3  while t < t_vnd and r ≤ r_max do
 4      set time_limit = min{t_mip, t_vnd − t};
 5      Q = (Q|{δ(x', x) ≤ r});
 6      x'' = MIPsolve(Q, time_limit, x');
 7      switch solution status do
 8          case OptSolFound:
 9              Reverse last pseudo-cut into δ(x', x) > r + 1;
10              x' = x'', r = 1;
11          case feasibleSolFound:
12              Replace last pseudo-cut with δ(x', x) ≥ 1;
13              x' = x'', r = 1;
14          case ProvenInfeasible:
15              Reverse last pseudo-cut into δ(x', x) > r + 1;
16              r = r + 1;
17          case nofeasiblesolfound:
18              return x';
        endsw
19      set t_end = CpuTime(), t = t_end − t_start;
    end
20  return x';
```

4.6.2 Hybrid Variable Neighborhood decomposition search heuristics

In 2010, Lazic et al. [38] proposed a hybrid heuristic for solving 0-1 mixed integer programs which combines variable neighborhood decomposition search (VNDS) with the CPLEX MIP solver (see Algorithm 23). The algorithm starts solving the LP-relaxation of the original problem obtaining an optimal solution $\bar{x}$. If the optimal solution $\bar{x}$ is integer feasible the procedure returns $\bar{x}$ as an optimal solution of the initial problem. Otherwise, an initial feasible solution x is generated. At each

iteration of the VNDS procedure, the distances $\delta_j = |x_j - \bar{x}_j|$ between the current incumbent solution values and corresponding LP-relaxation solution values are computed. Those distance values serve as criteria of choosing variables which will be fixed. Namely, at each iteration, k variables whose indices correspond to the indices of k smallest δ_j values, are fixed at their values in the current incumbent solution x. After that, the resulting problem is solved using the CPLEX MIP solver. If an improvement of the current solution is achieved, a Variable Neighborhood Descent branching is launched as the local search in the whole solution space and the process is repeated. If not, the number of fixed variables in the current subproblem is decreased. The pseudo-code is given in Algorithm 23. The input parameters for the VNDS algorithm are: the MIP problem P; the parameter d, which controls the change of neighborhood size during the search process; parameters $t_{max}, t_{sub}, t_{vnd}, t_{mip}, r_{max}$ which represent the maximum running time allowed for VNDS, time allowed for solving subproblems, time allowed for call to the `VND-MIP` procedure, time allowed for call to the MIP solver within the `VND-MIP` procedure, respectively. Finally, the parameter r_{max} represents maximum size of neighborhood to be explored within the `VND-MIP` procedure. In the pseudo-code the statement of the form $y =$ `FindFirstFeasible`(P) denotes a call to a generic MIP solver, an attempt to find a first feasible solution of an input problem P. Further, the statement of the form $y =$ `MIPsolve`(P, t, x^*) denotes a call to a generic MIP solver to solve input problem P within a given time limit t starting from the best solution found x^*.

Algorithm 23: Variable neighborhood decomposition search for 0-1 MIP

```
   Function VNDS(P, d, t_max, t_sub, t_vnd, t_mip, r_max);
 1 Solve the LP relaxation of P to obtain an optimal LP basic solution x̄;
 2 if x̄ MIP feasible then return x̄;
 3 x* = FindFirstFeasible(P);
 4 set t_start = CpuTime(); t = 0;
 5 while t < t_max do
 6 |   δ_j =| x*_j − x̄_j |; index x_j so that δ_j ≤ δ_{j+1}, j = 1, ..., |I| − 1;
 7 |   set q =| {j ∈ I | δ_j ≠ 0} |;
 8 |   set k_step = near(q/d), k = |I| − k_step;
 9 |   while t < t_max and k > 0 do
10 |   |   x' = MIPsolve(P(x̃, {1, ..., k}), t_sub, x*) ;
11 |   |   if cx' < cx* then
12 |   |   |   x* = VND-MIP(P, t_vnd, t_mip, r_max, x');
13 |   |   |   break;
   |   |   else
14 |   |   |   if k − k_step > |I| − q then k_step = max{near(k/2), 1};
15 |   |   |   set k = k − k_step;
16 |   |   |   set t_end = CpuTime(), t = t_end − t_start;
   |   |   end
   |   end
   end
17 return x*;
```

In 2010, Hanafi et al. [26] proposed a hybrid Variable Neighborhood decomposition search heuristic that constitutes an improved version of Variable Neighborhood decomposition search heuristic proposed in [38]. The enhancement is achieved by restricting the search space by adding pseudo cuts, in order to avoid multiple explorations of the same areas. A sequence of lower and upper bounds on the problem objective is produced by adding pseudo-cuts, thereby reducing the integrality gap.

4.6.3 Variable Neighborhood Pump

In 2010, Hanafi et al. [25] proposed a new method for finding an initial feasible solution for Mixed integer programs called *Variable Neighborhood Pump* (VNP) (Algorithm 24), that combines Variable neighborhood branching (VNB) [31] and Feasibility pump heuristics [19]. The VNP works in the following way. First, an optimal solution $\overline{x}$ of the LP-relaxation of the initial 0-1 MIP problem is determined. After that, the obtained solution is rounded and one iteration of the FP pumping cycle is applied on it in order to obtain a near-feasible vector $\tilde{x}$. Then on the solution $\tilde{x}$, variable neighborhood branching, adapted for 0-1 MIP feasibility [25], is applied, in an attempt to locate a feasible solution of the original problem. If VNB does not return a feasible solution a pseudo-cut is added to the current subproblem in order to change the linear relaxation solution, and the process is iterated. VNB returns either a feasible solution or reports failure and returns the last integer (infeasible) solution.

Algorithm 24: Variable Neighborhood Pump for 0-1 MIP

Function VNP(P);
Set $proceed1$ = `true`;
while $proceed1$ **do**
  Solve the LP relaxation of P to obtain an optimal LP basic solution $\overline{x}$;
  Set $\tilde{x} = near(\overline{x})$;
  Set $proceed2$ = `true`;
  while $proceed2$ **do**
    if $\overline{x}$ *is integer* **then return** $\overline{x}$;
    Solve the $LP(P, \tilde{x})$ problem to obtain an optimal solution $\overline{x}$;
    if $\tilde{x} \neq near(\overline{x})$ **then**
      $\tilde{x} = near(\overline{x})$;
    else
      Set $proceed2$ = `false`;
    end
  end
  $k_{min} = \lfloor \delta(\tilde{x}, \overline{x}) \rfloor$; $k_{max} = \lfloor (|\mathcal{I}| - k_{min})/2 \rfloor$; $k_{step} = (k_{max} - k_{min})/5$;
  $x' =$ VNB$(P, \tilde{x}, k_{min}, k_{step}, k_{max})$;
  if $x' = \tilde{x}$ **then**
    $P = (P \mid \delta(x, \overline{x}) \geq k_{min})$; Update $proceed1$;
  else
    return x';
  end
end
Output message: "No feasible solution found"; **return** $\tilde{x}$;

4.6.4 Diving heuristics

In 2014, Lazic et al. [39] proposed two diving heuristics for obtaining a first MIP feasible solution. Diving heuristics are based on the systematic hard variable fixing (diving) process, according to the information obtained from the linear relaxation solution of the problem. They rely on the observation that a general-purpose MIP solver can be used not only for finding (near) optimal solutions of a given input problem, but also for finding the initial feasible solution.

The variable neighborhood (VN) diving algorithm begins by obtaining the LP-relaxation solution $\overline{x}$ of the original problem P and generating an initial integer (not necessarily feasible) solution $\tilde{x} = near(\overline{x})$ by rounding the LP-solution $\overline{x}$. If the optimal solution $\overline{x}$ is integer feasible for P, VN diving stops and returns $\overline{x}$. At each iteration of the VN diving procedure, the distances $\delta_j = \mid \tilde{x}_j - \overline{x}_j \mid$ from the current integer solution values $(\tilde{x}_j)_{j \in \mathcal{I}}$ to the corresponding LP-relaxation solution values $(\overline{x}_j)_{j \in \mathcal{I}}$ are computed and the variables $\tilde{x}_j, j \in \mathcal{I}$ are indexed so that $\delta_1 \leq \delta_2 \leq \ldots \leq \delta_{|\mathcal{I}|}$. Then, VN diving successively solve the subproblems $P(\tilde{x}, \{1, \ldots, k\})$ obtained from the original problem P, where the first k variables are fixed to their values in the current incumbent solution $\tilde{x}$. If a feasible solution is found by solving $P(\tilde{x}, \{1, \ldots, k\})$, it is returned as a feasible solution of the original problem P. Otherwise, a pseudo-cut $\delta(\{1, \ldots, k\}, \tilde{x}, x) \geq 1$ is added in order to avoid exploring the search space of $P(\tilde{x}, \{1, \ldots, k\})$ again, and the next subproblem is examined. If no feasible solution is detected after solving all subproblems $P(\tilde{x}, \{1, \ldots, k\})$, $k_{min} \leq k \leq k_{max}$, $k_{min} = k_{step}$, $k_{max} = |\mathcal{I}| - k_{step}$, the linear relaxation of the current problem P, which includes all the pseudo-cuts added during the search process, is solved and the process is iterated. If no fea-

sible solution has been found due to the fulfillment of the stopping criteria, the algorithm reports failure and returns the last (infeasible) integer solution.

The pseudo-code of the VN diving heuristic is given in the Algorithm 25. The input parameters for the VN diving algorithm are the input MIP problem P and the parameter d, which controls the change of neighborhood size during the search process. In the pseudo-code the statement of the form $y =$ `FindFirstFeasible`(P, t) denotes a call to a generic MIP solver, an attempt to find a first feasible solution of an input problem P within a given time limit t. If a feasible solution is found, it is assigned to the variable y, otherwise y retains its previous value.

Algorithm 25: Variable neighborhood diving for 0-1 MIP feasibility.

Function `VNdiving`(P, d);
1 Set *proceed*1 = `true`, *proceed*2 = `true`; Set *timeLimit* for subproblems;
2 **while** *proceed*1 **do**
3 Solve the LP relaxation of P to obtain an optimal LP basic solution $\overline{x}$;
4 $\tilde{x} = near(\overline{x})$;
5 **if** $\overline{x} = \tilde{x}$ **then return** $\tilde{x}$;
6 $\delta_j =\mid \tilde{x}_j - \overline{x}_j \mid$; index x_j so that $\delta_j \leq \delta_{j+1}$, $j = 1, \ldots, |\mathcal{I}| - 1$;
7 Set $n_d =\mid \{j \in \mathcal{I} \mid \delta_j \neq 0\} \mid$, $k_{step} = near(n_d/d)$, $k = |\mathcal{I}| - k_{step}$;
8 **while** *proceed*2 **and** $k \geq 0$ **do**
9 $J_k = \{1, \ldots, k\}$; $x' =$ `FindFirstFeasible`$(P(\tilde{x}, J_k), timeLimit)$;
10 **if** $P(\tilde{x}, J_k)$ *is proven infeasible* **then** $P = (P \mid \delta(J_k, \tilde{x}, x) \geq 1)$;
11 **if** x' is feasible **then return** x';
12 **if** $k - k_{step} > |\mathcal{I}| - n_d$ **then** $k_{step} = \max\{near(k/2), 1\}$;
13 Set $k = k - k_{step}$;
14 Update *proceed*2;
  end
15 Update *proceed*1;
end
16 Output message: "No feasible solution found"; **return** $\tilde{x}$;

In the case of variable neighborhood diving, a set of subproblems $P(\tilde{x}, J_k)$, for different values of k, is examined in each iteration until a feasible solution is found. In the single neighborhood diving procedure, we only examine one subproblem $P(\tilde{x}, J_k)$ in each iteration (a single neighborhood, see Algorithm 26). However, because only a single neighborhood is examined, additional diversification mechanisms are required. This diversification is provided through keeping the list of constraints which ensures that the same reference integer solution $\widetilde{x}$ cannot occur more than once (i.e., in more than one iteration) in the solution process. An additional MIP problem Q is introduced to store these constraints. In the beginning of the algorithm, Q is initialized as an empty problem (see line 4 in Algorithm 26). Then, in each iteration, if the current reference solution $\widetilde{x}$ is not feasible (see line 8 in Algorithm 26), constraint $\delta(\widetilde{x}, x) \geq \lceil \delta(\widetilde{x}, \overline{x}) \rceil$ is added to Q (line 9). This guarantees that future reference solutions can not be the same as the current one, since the next reference solution is obtained by solving the problem MIP$(Q, near(\overline{x}))$ (see line 17), which contains all constraints from Q, (see definition (6)). The variables to be fixed in the current subproblem are chosen among those which have the same value as in the linear relaxation solution of the modified problem LP$(\widetilde{x})$, where $\widetilde{x}$ is the current reference integer solution (see lines 7 and 11). The number of variables to be fixed is controlled by the parameter α (line 11). After initialization (line 5), the value of α is updated in each iteration, depending on the solution status returned from the MIP solver. If the current subproblem is proven infeasible,

the value of α is increased in order to reduce the number of fixed variables in the next iteration (see line 16), and thus provide better diversification. Otherwise, if the time limit allowed for subproblem is exceeded without reaching a feasible solution or proving the subproblem infeasibility, the value of α is decreased. Decreasing the value of α, increases the number of fixed variables in the next iteration (see line 17), and thus reduces the size of the next subproblem. In the feasibility pump, the next reference integer solution is obtained by simply rounding the linear relaxation solution $\overline{x}$ of the modified problem LP($\widetilde{x}$). However, if $near(\overline{x})$ is equal to some of the previous reference solutions, the solution process is caught in a cycle. In order to avoid this type of cycling, we determine the next reference solution as the one which is at the minimum distance from $near(\overline{x})$ (with respect to binary variables) and satisfies all constraints from the current subproblem Q (see line 18). In this way the convergence of the variable neighborhood diving algorithm is guaranteed (see [39]).

Algorithm 26: Single neighborhood diving for 0-1 MIP feasibility.

Function `SNDiving`(P);
1 Solve the LP relaxation of P to obtain an optimal LP basic solution $\overline{x}$;
2 Set $i = 0$; Set $\widetilde{x}^0 = near(\overline{x})$;
3 **if** $(\overline{x} = \widetilde{x}^0)$ **then return** $\widetilde{x}^0$;
4 Set $Q_0 = \emptyset$;
5 Set *proceed* = `true`; Set *timeLimit* for subproblems; Set value of α;
6 **while** *proceed* **do**
7 Solve the $LP(P, \widetilde{x}^i)$ problem to obtain an optimal solution $\overline{x}$;
8 **if** $(\lceil \delta(\widetilde{x}^i, \overline{x}) \rceil = 0)$ **then return** $\widetilde{x}^i$;
9 $Q_{i+1} = (Q_i \mid \delta(\widetilde{x}^i, x) \geq \lceil \delta(\widetilde{x}^i, \overline{x}) \rceil)$;
10 $\delta_j = \mid \widetilde{x}^i_j - \overline{x}_j \mid$; index x_j so that $\delta_j \leq \delta_{j+1}$, $j = 1, \ldots, |\mathcal{I}| - 1$;
11 $k = near(\mid \{j \in \mathcal{I} : \widetilde{x}^i_j = \overline{x}_j\} \mid /\alpha)$; $J_k = \{1, \ldots, k\}$;
12 $x' =$ `FindFirstFeasible`$(P(\widetilde{x}^i, J_k), timeLimit)$;
13 **if** *feasible solution found* **then return** x';
14 **if** $P(\widetilde{x}^i, J_k)$ is proven infeasible **then**
15 $Q_{i+1} = (Q_{i+1} \mid \delta(J_k, \widetilde{x}^i, x) \geq 1)$; $P = (P \mid \delta(J_k, \widetilde{x}^i, x) \geq 1)$;
16 $\alpha = 3\alpha/2$;
else
17 **if** *time limit for subproblem exceeded* **then** $\alpha = \max\{1, \alpha/2\}$;
end
18 $\widetilde{x}^{i+1} =$ `FindFirstFeasible`$(\text{MIP}(Q_{i+1}, near(\overline{x})), timeLimit)$;
19 **if** $\text{MIP}(Q_{i+1}, near(\overline{x}))$ *is proven infeasible* **then** Output message: "Problem P is proven infeasible"; **return**;
20 $i = i + 1$;
end

Algorithms 22 to 26 may look at first glance that they do not necessary have VNS nature and therefore do not have strong connection with algorithms from sections 2 and 3. However, it should be noted that those methods all systematically use different neighborhood structures induced from the Hamming distance. In Algorithm 22, the parameter r in fact represents the neighborhood index of VND algorithm from before. The same neighborhoods are used in Algorithm 23, where the size of sub-problems is larger and larger, and the number of fixed variables is smaller and smaller. This is in fact VNDS from before, adapted for solving a 0-1 MIP problem using general purpose solvers such as CPLEX, Gurobi etc. A similar analogy holds for Algorithms 24-26.

4.7 Parallel VNS

Parallel VNS heuristics are usually applied for solving large instances of optimization problems in a reasonable time. In the literature various parallelization strategies within VNS heuristics have been performed for this purpose. Besides straightforward parallelization strategies (e.g., (i) parallelize local search; (ii) augment the number of solutions drawn from the current neighborhood and make a local search in parallel from each of them; or (iii) do the same as (ii) but update the information about the best solution found) several more sophisticated approaches that lead to much better performance have been proposed (see e.g., [13]). These parallel VNS heuristics were used to solve many combinatorial optimization problems (e.g., p-median problem, car sequencing problem, vehicle routing related problems, job shop scheduling problems, dynamic memory allocation problems etc.) [12, 22, 37, 55, 56, 59, 60, 62, 69] as well as the multi-objective optimization problems (see e.g., [18]).

4.8 Hybrids

The VNS has been also hybridized with other metaheuristics (e.g., tabu search, path relinking, simulated annealing, greedy randomized adaptive search procedures (GRASP), genetic algorithm, particle swarm optimization etc). For example, Li, et al. [40] combined the VNS with the chemical-reaction optimization (CRO) and the estimation of distribution (EDA) to solve the hybrid flow shop scheduling problem. In [53], General Variable Neighborhood Search was hybridized with GRASP to solve the targeted offers problem in direct marketing campaigns. Belhaiza et al. proposed a hybrid variable neighborhood-tabu search heuristic for the vehicle routing problem with multiple time windows [5], while Xiao et al. used variable neighborhood simulated annealing algorithm to solve capacitated vehicle routing problems. For other VNS hybridization we refer the reader to [29].

5 Conclusions

The basic scheme of variable neighborhood search (VNS) along with its main ingredients (local search and shaking procedures) have been presented. More precisely, we present most common local search procedures used within a VNS heuristic as well as a typical shaking procedure to resolve local optima traps. In addition, the paper contains VNS variants that have been deduced from the basic VNS scheme.

VNS based heuristics turn out to be the state-of-the-art heuristics for many NP-hard optimization problems. Such performance indicates that developing VNS heuristics for solving other NP-hard optimization problems will lead to a promising research avenue. Thus, we believe that this paper will serve as a guide for developing new state-of-the-art heuristics based on VNS.

References

1. de Armas, J., Melián-Batista, B.: Variable neighborhood search for a dynamic rich vehicle routing problem with time windows. Computers & Industrial Engineering **85**, 120–131 (2015)
2. Audet, C., Béchard, V., Le Digabel, S.: Nonsmooth optimization through mesh adaptive direct search and variable neighborhood search. Journal of Global Optimization **41**(2), 299–318 (2008)
3. Audet, C., Brimberg, J., Hansen, P., Digabel, S.L., Mladenović, N.: Pooling problem: Alternate formulations and solution methods. Management science **50**(6), 761–776 (2004)
4. Balas, E., Martin, C.: Pivot and complement-a heuristic for 0–1 programming. Managment Science **26**, 86–96 (1980)
5. Belhaiza, S., Hansen, P., Laporte, G.: A hybrid variable neighborhood tabu search heuristic for the vehicle routing problem with multiple time windows. Computers & Operations Research **52**, 269–281 (2014)
6. Bierlaire, M., Thémans, M., Zufferey, N.: A heuristic for nonlinear global optimization. INFORMS Journal on Computing **22**(1), 59–70 (2010)
7. Brimberg, J., Hansen, P., Mladenovic, N., Taillard, E.D.: Improvements and comparison of heuristics for solving the uncapacitated multisource weber problem. Operations Research **48**(3), 444–460 (2000)
8. Brimberg, J., Mladenović, N., Urošević, D.: Solving the maximally diverse grouping problem by skewed general variable neighborhood search. Information Sciences **295**, 650–675 (2015)
9. Bussieck, M.R., Drud, A.S., Meeraus, A.: Minlplib – a collection of test models for mixed-integer nonlinear programming. INFORMS Journal on Computing **15**(1), 114–119 (2003)
10. Carrizosa, E., Dražić, M., Dražić, Z., Mladenović, N.: Gaussian variable neighborhood search for continuous optimization. Computers & Operations Research **39**(9), 2206–2213 (2012)
11. Carrizosa, E., Mladenović, N., Todosijević, R.: Variable neighborhood search for minimum sum-of-squares clustering on networks. European Journal of Operational Research **230**(2), 356–363 (2013)
12. Crainic, T.G., Gendreau, M., Hansen, P., Mladenović, N.: Cooperative parallel variable neighborhood search for the p-median. Journal of Heuristics **10**(3), 293–314 (2004)
13. Davidovic, T., Crainic, T.G.: Parallelization strategies for variable neighborhood search. Tecnical report CIRRELT—2013–47 (2013)
14. Di Gaspero, L., Schaerf, A.: Neighborhood portfolio approach for local search applied to timetabling problems. Journal of Mathematical Modelling and Algorithms **5**(1), 65–89 (2006)
15. Dražić, M., Lavor, C., Maculan, N., Mladenović, N.: A continuous variable neighbourhood search heuristic for finding the tridimensional structure of a molecule. European Journal of Operational Research **185**, 1265–1273 (2008)
16. Dražić, M., Dražić, Z., Mladenović, N., Urošević, D., Zhao, Q.H.: Continuous variable neighbourhood search with modified nelder–mead for non-differentiable optimization. IMA Journal of Management Mathematics, doi: 10.1093/imaman/dpu012

17. Drazić, M., Kovacevic-Vujcić, V., Cangalović, M., Mladenović, N.: Glob – a new vns-based software for global optimization. In: Global optimization, pp. 135–154. Springer (2006)
18. Eskandarpour, M., Zegordi, S.H., Nikbakhsh, E.: A parallel variable neighborhood search for the multi-objective sustainable post-sales network design problem. International Journal of Production Economics **145**(1), 117–131 (2013)
19. Fischetti, M., Glover, F., Lodi, A.: The feasibility pump. Mathematical Programming **104**(1), 91–104 (2005)
20. Fischetti, M., Lodi, A.: Local branching. Mathematical Programming **98**, 23–47 (2003)
21. Fletcher, R., Leyffer, S.: Numerical experience with lower bounds for miqp branch-and-bound. SIAM Journal on Optimization **8**(2), 604–616 (1998)
22. García-López, F., Melián-Batista, B., Moreno-Pérez, J.A., Moreno-Vega, J.M.: The parallel variable neighborhood search for the p-median problem. Journal of Heuristics **8**(3), 375–388 (2002)
23. Gill, P.E., Murray, W., Saunders, M.A.: Snopt: An sqp algorithm for large-scale constrained optimization. SIAM journal on optimization **12**(4), 979–1006 (2002)
24. Glover, F., McMillan, C., Glover, R.: A heuristic programming approach to the employee scheduling problem and some thoughts on managerial robots. Journal of Operations Management **4**(2), 113–128 (1984)
25. Hanafi, S., Lazić, J., Mladenović, N.: Variable neighbourhood pump heuristic for 0–1 mixed integer programming feasibility. Electronic Notes in Discrete Mathematics **36**, 759–766 (2010)
26. Hanafi, S., Lazić, J., Mladenović, N., Wilbaut, C., Crévits, I.: Hybrid variable neighbourhood decomposition search for 0–1 mixed integer programming problem. Electronic Notes in Discrete Mathematics **36**, 883–890 (2010)
27. Hansen, P., Brimberg, J., Uroševic, D., Mladenovic, N.: Primal-dual variable neighborhood search for the simple plant-location problem. INFORMS Journal on Computing **19**(4), 552–564 (2007)
28. Hansen, P., Mladenović, N.: First vs. best improvement: An empirical study. Discrete Applied Mathematics **154**(5), 802–817 (2006)
29. Hansen, P., Mladenović, N., Pérez, J.A.M.: Variable neighbourhood search: methods and applications. Annals of Operations Research **175**(1), 367–407 (2010)
30. Hansen, P., Mladenović, N., Perez-Britos, D.: Variable neighborhood decomposition search. Journal of Heuristics **7**(4), 335–350 (2001)
31. Hansen, P., Mladenović, N., Urosević, D.: Variable neighborhood search and local branching. Computers & Operations Research **33**(10), 3034–3045 (2006)
32. Ilić, A., Urošević, D., Brimberg, J., Mladenović, N.: A general variable neighborhood search for solving the uncapacitated single allocation p-hub median problem. European Journal of Operational Research **206**(2), 289–300 (2010)
33. Johnson, D.S., McGeoch, L.: The traveling salesman problem: a case study in local optimization. In: E. Aarts, J. Lenstra (eds.) Local Search in Combinatorial Optimization, pp. 215–310. John Wiley and Sons, New York (1997)
34. Khemakhem, M., Haddar, B., Chebil, K., Hanafi, S.: A filter-and-fan metaheuristic for the 0-1 multidimensional knapsack problem. Int. J. of Applied Metaheuristic Computing **3**, 43–63 (2012)

35. Kirkpatrick, S., Toulouse, G.: Configuration space analysis of traveling salesman problems. Journal de physique **46**, 1277–1292 (1985)
36. Kirlik, G., Oguz, C.: A variable neighborhood search for minimizing total weighted tardiness with sequence dependent setup times on a single machine. Computers & Operations Research **39**(7), 1506–1520 (2012)
37. Knausz, M.: Parallel variable neighbourhood search for the car sequencing problem. Tecnical report, Fakultat fur Informatik der Technischen Universitat Wien (2008)
38. Lazić, J., Hanafi, S., Mladenović, N., Urosević, D.: Variable neighbourhood decomposition search for 0–1 mixed integer programs. Computers & Operations Research **37**, 1055–1067 (2010)
39. Lazić, J., Todosijević, R., Hanafi, S., Mladenović, N.: Variable and single neighbourhood diving for mip feasibility. Yugoslav Journal of Operations Research (2014). DOI 10.2298/YJOR140417027L
40. Li, J.q., Pan, Q.k., Wang, F.t.: A hybrid variable neighborhood search for solving the hybrid flow shop scheduling problem. Applied Soft Computing **24**, 63–77 (2014)
41. Liberti, L., Lavor, C., Maculan, N., Marinelli, F.: Double variable neighbourhood search with smoothing for the molecular distance geometry problem. Journal of Global Optimization **43**, 207–218 (2009)
42. Liberti, L., Mladenović, N., Nannicini, G.: A recipe for finding good solutions to MINLPs. Mathematical Programming Computation **3**, 349–390 (2011)
43. Liberti, L., Nannicini, G., Mladenović, N.: A good recipe for solving minlps. In: V. Maniezzo, T. Stuetze, S. Voss (eds.) MATHEURISTICS: Hybridizing metaheuristics and mathematical programming, Operations Research/Computer Science Interface Series, pp. 231–244. Springer, Berlin (2010)
44. Lü, Z., Hao, J.K., Glover, F.: Neighborhood analysis: a case study on curriculum-based course timetabling. Journal of Heuristics **17**(2), 97–118 (2011)
45. Mjirda, A., Todosijevic, R., Hanafi, S., Hansen, P., Mladenović, N.: Sequential variable neighborhood descent variants: An empirical study on travelling salesman problem. working paper
46. Mladenović, N., Dražić, M., Kovačevic-Vujčić, V., Čangalović, M.: General variable neighborhood search for the continuous optimization. European Journal of Operational Research **191**(3), 753–770 (2008)
47. Mladenović, N., Hansen, P.: Variable neighborhood search. Computers & Operations Research **24**(11), 1097–1100 (1997)
48. Mladenović, N., Petrović, J., Kovačević-Vujčić, V., Čangalović, M.: Solving spread spectrum radar polyphase code design problem by tabu search and variable neighbourhood search. European Journal of Operational Research **151**(2), 389–399 (2003)
49. Mladenović, N., Plastria, F., Urošević, D.: Reformulation descent applied to circle packing problems. Computers & Operations Research **32**(9), 2419–2434 (2005)
50. Mladenović, N., Todosijević, R., Urošević, D.: An efficient general variable neighborhood search for large travelling salesman problem with time windows. Yugoslav Journal of Operations Research **23**(1), 19–31 (2013)
51. Mladenović, N., Todosijević, R., Urošević, D.: Two level general variable neighborhood search for attractive traveling salesman problem. Computers and

Operations Research **52**, 341–348 (2014)
52. Mladenović, N., Urošević, D., Perez-Brito, D.: Variable neighborhood search for minimum linear arrangement problem. Yugoslav Journal of Operations Research (2014). DOI 10.2298YJOR140928038M
53. Oliveira, T., Coelho, V., Souza, M.J.F., Boava, D.L.T., Boava, F., Coelho, I.M., Coelho, B.N.: A hybrid variable neighborhood search algorithm for targeted offers in direct marketing. Electronic Notes in Discrete Mathematics **47**, 205–212 (2015)
54. Pardo, E.G., Mladenović, N., Pantrigo, J.J., Duarte, A.: Variable formulation search for the cutwidth minimization problem. Applied Soft Computing **13**(5), 2242–2252 (2013)
55. Pirkwieser, S., Raidl, G.R.: Multiple variable neighborhood search enriched with ilp techniques for the periodic vehicle routing problem with time windows. In: Hybrid Metaheuristics, pp. 45–59. Springer (2009)
56. Polacek, M., Benkner, S., Doerner, K.F., Hartl, R.F.: A cooperative and adaptive variable neighborhood search for the multi depot vehicle routing problem with time windows. BuR-Business Research **1**(2), 207–218 (2008)
57. Rego, C., Glover, F.: Ejection chain and filter-and-fan methods in combinatorial optimization. Annals of Operations Research **175**(1), 77–105 (2010)
58. Reinelt, G.: TSPLIB - a traveling salesman problem library. ORSA Journal on Computing **3**, 376–384 (1991)
59. Sánchez-Oro, J., Sevaux, M., Rossi, A., Martí, R., Duarte, A.: Solving dynamic memory allocation problems in embedded systems with parallel variable neighborhood search strategies. Electronic Notes in Discrete Mathematics **47**, 85–92 (2015)
60. Sevkli, M., Aydin, M.E.: Parallel variable neighbourhood search algorithms for job shop scheduling problems. IMA Journal of Management Mathematics **18**(2), 117–133 (2007)
61. Todosijević, R., Benmanosur, R., Hanafi, S., Mladenović, N., Artiba, A.: Nested general variable neighborhood search for the periodic maintenance problem. European Journal of Operational Research, submitted (2014)
62. Todosijević, R., Hanafi, S., Urošević, D., Jarboui, B., Gendron, B.: Variable neighborhood search based heuristic for the swap-body vehicle routing problem. Computers and Operations Research (submitted) (2015)
63. Todosijević, R., Mjirda, A., Mladenović, M., Hanafi, S., Gendron, B.: A general variable neighborhood search variants for the travelling salesman problem with draft limits. Optimization Letters (2014). DOI 10.1007/s11590-014-0788-9
64. Todosijević, R., Mladenović, M., Hanafi, S., Crévits, I.: Vns based heuristic for solving the unit commitment problem. Electronic Notes in Discrete Mathematics **39**, 153–160 (2012)
65. Todosijević, R., Mladenović, M., Hanafi, S., Mladenović, N., Crévts, I.: Adaptive general variable neighborhood search heuristics for solving unit commitment problem. International Journal of Electrical Power and Energy Systems, submitted (2014)
66. Todosijević, R., Urošević, D., Mladenović, N., Hanafi, S.: A general variable neighborhood search for solving the uncapacitated r-allocation p-hub median problem. Optimization Letters (2015). DOI 10.1007/s11590-015-0867-6
67. Toksarı, M.D., Güner, E.: Solving the unconstrained optimization problem by a variable neighborhood search. Journal of Mathematical Analysis and

Applications **328**(2), 1178–1187 (2007)
68. Wu, Q., Hao, J.K., Glover, F.: Multi-neighborhood tabu search for the maximum weight clique problem. Annals of Operations Research **196**(1), 611–634 (2012)
69. Yazdani, M., Amiri, M., Zandieh, M.: Flexible job-shop scheduling with parallel variable neighborhood search algorithm. Expert Systems with Applications **37**(1), 678–687 (2010)
70. Zhao, Q., Mladenovic, N., Uroševic, D.: A parametric simplex search for unconstrained optimization problem. Transactions on Advanced Research **8**, 22–27 (2012)
71. Zhao, Q.H., Urosević, D., Mladenović, N., Hansen, P.: A restarted and modified simplex search for unconstrained optimization. Computers & Operations Research **36**(12), 3263–3271 (2009)